OS Lab Assignment 7 — Threads, Semaphores, Deadlock Detection and Priority Scheduling in xv6

Name: Priyam Das **Roll Number:** 2401MC04

How to Build and Run

```
make qemu-nox       # 2 CPUs (Q1-Q3 and the 2-CPU Q4 run)
make qemu-nox CPUS=1  # 1 CPU (main Q4 run)
```

Inside xv6, start `threadtest`, `synctest`, `deadlock` and `priotest`. The terminal output of every run is saved under `output_logs/`.

Folder	Contents
kernel/	`proc.c, proc.h, sysproc.c, syscall.c, syscall.h, defs.h, param.h, trap.c`
user/	`threadtest.c, synctest.c, deadlock.c, priotest.c, user.h, usys.S`
Makefile	registers the four new programs in UPROGS, carries the `-smp N,sockets=N` fix and no longer passes `-Werror`
output_logs/	one terminal log per question plus the complete 2-CPU session

New system calls: `clone` (26), `join` (27), `set_priority` (28), `get_priority` (29). `sem_init` (23) and `sem_wait` (24) were rewritten, and `sem_signal` (25) takes the place of A5's `sem_post`.

Q1 — Kernel-Supported Threads (clone / join)

Design and data structures. `struct proc` now carries an `isthread` flag. `clone(fcn, arg, stack)` is built like `fork()`: `allocproc()` supplies a new PCB and kernel stack, but `np->pgdir` is set to `curproc->pgdir`, so the address space is shared rather than copied. A fake return address `0xffffffff` and then `arg` are pushed onto the user stack that the caller provided; `tf->esp` points at that frame and `tf->eip` is set to `fcn`. Open files and the cwd are duplicated exactly as `fork()` does. `sys_clone` fetches the stack with `argptr(2, &stack;, PGSIZE)`, so the kernel refuses any stack that is not completely inside the process's own memory.

`join()` follows the structure of `wait()` but only collects children whose `isthread == 1`. It releases the kernel stack and **never calls freevm**, because the page directory is still owned by the parent.

Changes to exit / wait / fork:

- `wait()` ignores thread children, so a normal wait cannot free a shared pgdir by accident. It calls `freevm()` only if `!p->isthread`. `init` is the exception: it may reap orphaned threads, and does so without `freevm`.
- `exit()` hands any still-running threads of the exiting process to `init` after marking them `killed`.
- `fork()` invoked from a thread copies the shared pgdir through `copyuvm`, so no change was needed there.

Result before synchronization (3 threads x 100,000 increments, 2 CPUs):

Run	Final counter	Lost updates
1	271,794	28,206
2	211,379	88,621
3	223,534	76,466

Every thread's increments show up in the parent's own `counter`, which confirms the address space is really shared. The lost updates come from the non-atomic load / add / store sequence on a `volatile int` when two CPUs run it at once.

Q2 — Counting Semaphores

Design and data structures. Both structures live in `proc.c`:

```
struct semaphore { int value; int wakeups; struct spinlock lk; };
struct { struct spinlock lock; struct semaphore sem[NSEM]; } semtable; /* NSEM = 10 */
```

`pinit()` sets up all the locks. `sem_init` takes the table lock, then the semaphore's own lock. `sem_wait` decrements `value`; if the result is negative it calls `sleep(s, &s->lk)` until `wakeups > 0`, and then uses up one wakeup. `sem_signal` increments `value` and, when the result is <= 0, adds a wakeup and calls `wakeup(s)`. A negative `value` therefore equals the number of blocked processes, as the specification says. The `wakeups` counter makes sure only one sleeper proceeds per signal, even though xv6's `wakeup()` wakes every process sleeping on the channel.

Before/after comparison of the race condition:

Program	Synchronization	Final counter
`threadtest`	none	211,379 - 271,794 (wrong, varies per run)
`synctest` part 1	`sem_wait(0)` / `sem_signal(0)` around `counter++`	**300,000** (exact)

Producer-Consumer (`synctest` part 2): a producer thread and a consumer thread share a circular buffer of 5 slots, guarded by `empty = 5`, `full = 0` and `mutex = 1`. Items are produced and consumed in FIFO order. The printed buffer count stayed between 0 and 5 throughout: the producer blocks when it reaches 5 and the consumer blocks at 0. Final result: `count=0, overflow=no, underflow=no`.

Q3 — Deadlock Detection and the Banker's Algorithm

Design. `deadlock.c` stores `avail[m]`, `alloc[n][m]`, `request[n][m]`, `max[n][m]` and `need[n][m]` as static arrays.

- `detect_deadlock(int *list)` initialises `Work = Available`, and marks `Finish[i] = true` for every process holding nothing. It then keeps searching for an unfinished process with `Request <= Work`, adds that process's allocation to `Work` and prints each step. Whichever processes are still unfinished at the end are deadlocked, and their indices go into `list`.
- `is_safe_state()` is the Banker's safety algorithm, working on `Need = Max - Allocation`. It prints either the safe sequence or UNSAFE.
- `request_resources(p, req)` verifies `req <= Need` and `req <= Available`, allocates tentatively, calls `is_safe_state()`, and undoes the allocation if the result is unsafe.
- Real deadlock: the parent creates two pipes and forks A and B. A keeps Pipe1's write end and blocks reading Pipe2; B keeps Pipe2's write end and blocks reading Pipe1. Each child tells the parent over a status pipe that it is about to block. The parent waits 100 ticks, builds the allocation and request matrices for the two pipes, runs detection, and finally kills both children.

Results:

Test	Matrix	Expected	Program output
1	5x3, Request taken from the textbook example	no deadlock	`no deadlock` (order P0, P2, P3, P4, P1)
2	same, but P2 requests one more C	deadlock	`DEADLOCK among P1 P2 P3 P4`
3	Banker's, Available = (3,3,2)	safe	`SAFE <P1 P3 P4 P0 P2>`; P1's request (1,0,2) **granted**
4	after test 3, P0 requests (0,2,0)	becomes unsafe	`UNSAFE`, request **denied** and rolled back
5	real processes A and B on 2 pipes	circular wait	`DEADLOCK (circular wait) among A B`

Available, Allocation and Request/Max/Need are printed at every step (see `q3_deadlock.txt`).

Q4 — Priority-Based Preemptive Scheduler with Aging

Design. `struct proc` gained `priority` (1 is highest, 20 is lowest; `allocproc` defaults it to 10) and `waitticks`.

- `scheduler()` walks the whole process table beginning right after the slot it picked last time (`last`, protected by `ptable.lock`) and runs the RUNNABLE process with the smallest priority number. Since it only switches to a strictly smaller number, ties are given to whichever process comes first after `last`, which produces round-robin among equals.
- Preemption relies on xv6's existing timer-driven `yield()`: on each tick the scheduler runs again and chooses the best process.
- Aging: `trap.c` calls `ageprocs()` on every timer tick, which increments `waitticks` for each RUNNABLE process. When a process reaches `AGE_TICKS = 30`, its priority number falls by 1 (never below 1) and its counter restarts. Getting scheduled also resets the counter.
- `set_priority(pid, prio)` verifies that `prio` lies between 1 and 20. `get_priority(pid)` returns -1 for an unknown pid.

`priotest` forks four identical CPU-bound children with priorities 2, 6, 12 and 20. The parent runs at priority 1 so that forking is never delayed. A monitor process, also at priority 1, polls `get_priority()` for the priority-20 child and prints every aging step.

Completion time vs. priority (`CPUS=1`, `q4_priotest_1cpu.txt`):

Rank	Start priority	Completion (ticks)	Final priority
1	2	147	2
2	6	293	2
3	12	439	3
4	20	581	6

The order of completion is identical to the priority order. The priority-20 process aged one level every 30-32 waiting ticks (20 to 19 at tick 34, ..., 7 to 6 at tick 436), then received the CPU and finished, so it was not starved.

On 2 CPUs (`q4_priotest_2cpu.txt`) the order is again 2, 6, 12, 20 (144 / 156 / 287 / 305 ticks). The two highest-priority jobs run side by side, and the last two get CPUs only after the first two are done.

Problems Faced and How They Were Resolved

1. **The race did not appear at first, and `cpu0` was the only CPU booting.** QEMU 10 converts `-smp 2` into a single socket with two cores, and the MP table that xv6 reads then lists just one CPU. Printing `ncpu` at boot showed 1, which exposed the problem. The Makefile now uses `-smp $(CPUS),sockets=$(CPUS)`, after which `cpu1: starting` appears.
2. **Threads ended before the next one started,** so their increments never overlapped. I added a `volatile int` go-start barrier that the parent releases once all threads exist.
3. **The build failed under GCC 15.** `-Werror` promoted the `-Warray-bounds` warnings in stock `mp.c` to errors. Removing `-Werror` fixed it, which matches how the previous assignments were built.
4. **Lost wakeups ("thundering herd").** xv6's `wakeup()` wakes every sleeper on a channel, so the semaphore keeps a `wakeups` counter and lets exactly one sleeper through for each signal.
5. `struct spinlock` **inside** `proc.h` **broke files that include** `proc.h` **before** `spinlock.h`**.** I placed `struct semaphore` in `proc.c` where only that file uses it.
6. `priotest` **hung after the first two jobs.** Each child looped on `sleep(1)` until its priority matched the assigned value. Aging changed the priority while the child was waiting, so the loop never terminated. I dropped this handshake: the priority-1 parent

now sets each child's priority right after `fork()`.

7. **Interleaved output from the two deadlock children** running on separate CPUs. The parent now forks B only after A has reported.

Known limitation: if a process exits while its threads are still running, the address space is released as soon as the parent is reaped, even though the threads are already marked killed. All the test programs `join()` their threads before exiting.